

CIS 5511 – Homework 3

Md. Mizanur Rahman
md.mizanur.rahman@temple.edu

September 24, 2026

Task 1

Data Structure Design

The priority queue is implemented using a binary min-heap. The heap stores English words and maintains them according to dictionary order. A min-heap is appropriate for this problem because the smallest word in lexicographical order is always stored at the root of the heap. Therefore, the first word in dictionary order can be accessed directly from the root.

Variables Used

The data structure uses the following variables:

- **A**: An array used to store the English words. In the C++ implementation, this array is represented using `vector<string>`.
- **heapSize**: An integer variable that stores the current number of words in the priority queue.

Index 0 is not used intentionally. Therefore, root is at index 1. This makes the index calculations identical to the standard heap representation discussed in the lecture.

For a node stored at index i , the index of its parent is

$$\text{Parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor.$$

The index of its left child is

$$\text{Left}(i) = 2i,$$

and the index of its right child is

$$\text{Right}(i) = 2i + 1.$$

Min-Heap Property

The priority queue satisfies the min-heap property. For every node except the root,

$$A[\text{Parent}(i)] \leq A[i].$$

Here, the comparison between two words is based on dictionary order. Therefore, every parent word appears before or is equal to its children in dictionary order.

Since duplicate words are not allowed in this data structure, the values stored in the heap are distinct. Thus, for the words stored in the priority queue, the parent normally appears before its children in dictionary order.

Because of the min-heap property, the word stored at the root,

$$A[1],$$

is always the first word in dictionary order among all words currently stored in the priority queue.

Example

Suppose the following five words are stored in the priority queue:

apple, banana, orange, pear, grape.

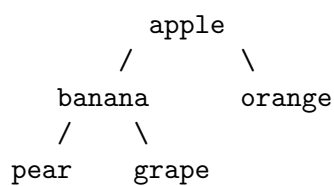
One possible min-heap representation is

$$A = [\text{apple}, \text{banana}, \text{orange}, \text{pear}, \text{grape}].$$

Since index 0 is unused, the actual indexes are

Index	1	2	3	4	5
Word	apple	banana	orange	pear	grape

The corresponding binary heap can be represented as



The root word **apple** is the first word in dictionary order.

The parent-child relationships also satisfy the min-heap property:

$$\text{apple} < \text{banana},$$

$$\text{apple} < \text{orange},$$

$$\text{banana} < \text{pear},$$

and

$$\text{banana} < \text{grape}.$$

Therefore, this is a valid min-heap.

Why a Min-Heap is Used

A normal max-heap keeps the largest value at the root. However, this assignment requires removing the first word in dictionary order. Therefore, a min-heap is appropriate.

Using a min-heap allows the first word in dictionary order to always be located at the root. As a result, accessing the word to be removed does not require searching through the entire data structure.

It is important to note that a heap is not a completely sorted array. The min-heap property only guarantees that each parent precedes its children in dictionary order. It does not guarantee an ordering between nodes in different subtrees. This fact becomes important when designing the search operation.

Task 2

Algorithms

Min-heap algorithm is used here, where the minimum element is stored at the root because the first word in dictionary order must be removed first.

The heap uses 1-based indexing. Therefore,

$$\text{PARENT}(i) = \left\lfloor \frac{i}{2} \right\rfloor,$$

$$\text{LEFT}(i) = 2i,$$

and

$$\text{RIGHT}(i) = 2i + 1.$$

The variable `A.heap-size` stores the number of words currently contained in the heap.

Min-Heapify

Algorithm 1 MIN-HEAPIFY(A, i)

```

1:  $l = \text{LEFT}(i)$ 
2:  $r = \text{RIGHT}(i)$ 
3: if  $l \leq A.\text{heap-size}$  and  $A[l] < A[i]$  then
4:    $\text{smallest} = l$ 
5: else
6:    $\text{smallest} = i$ 
7: end if
8: if  $r \leq A.\text{heap-size}$  and  $A[r] < A[\text{smallest}]$  then
9:    $\text{smallest} = r$ 
10: end if
11: if  $\text{smallest} \neq i$  then
12:   exchange  $A[i]$  with  $A[\text{smallest}]$ 
13:   MIN-HEAPIFY( $A, \text{smallest}$ )
14: end if
```

The algorithm first compares the word at position i with its left child. The smaller word is recorded in `smallest`. It then compares this word with the right child. If one of the children

contains a word that comes before the current word in dictionary order, the two words are exchanged.

After the exchange, the heap property may be violated in the corresponding subtree. Therefore, MIN-HEAPIFY is called recursively on that subtree.

For example, suppose the current heap contains

[grape, banana, orange, pear].

At the root, **banana** is smaller than **grape**. The two values are exchanged, producing

[banana, grape, orange, pear].

The min-heap property is then restored.

Report the Number of Words

The number of words currently stored in the priority queue is already maintained by **A.heap-size**. Therefore, the operation simply returns this value.

Algorithm 2 HEAP-SIZE(*A*)

1: **return** *A.heap-size*

No traversal of the heap is necessary because **A.heap-size** is updated whenever a word is added or removed.

Remove the First Word in Dictionary Order

Since the data structure is a min-heap, the root $A[1]$ always contains the first word in dictionary order.

First, the root is removed, then the last element of the heap is moved to the root, and the heap is repaired from the top down using MIN-HEAPIFY.

First, the minimum word can be obtained using the following algorithm.

Algorithm 3 MIN-HEAP-MINIMUM(*A*)

1: **if** *A.heap-size* < 1 **then**
 2: error “heap underflow”
 3: **end if**
 4: **return** $A[1]$

The removal operation is then performed as follows.

Algorithm 4 MIN-HEAP-EXTRACT-MIN(*A*)

1: $min = \text{MIN-HEAP-MINIMUM}(A)$
 2: $A[1] = A[A.heap-size]$
 3: $A.heap-size = A.heap-size - 1$
 4: **if** *A.heap-size* > 0 **then**
 5: MIN-HEAPIFY(*A*, 1)
 6: **end if**
 7: **return** *min*

For example, suppose the heap is

[apple, banana, orange, pear, grape].

The root **apple** is removed. The last word, **grape**, is moved to the root:

[grape, banana, orange, pear].

The min-heap property is now violated because

banana < grape.

After calling MIN-HEAPIFY, the heap becomes

[banana, grape, orange, pear].

Thus, **apple** is returned and the remaining words still form a valid min-heap.

Search for a Word

Unlike a sorted array, a heap is only partially ordered. Therefore, binary search cannot be used to search for an arbitrary word.

However, the min-heap property can still be used to avoid searching some subtrees. If the target word comes before the word stored at the current node, then the target cannot occur below that node. This is because every descendant must be greater than or equal to the current node.

The search algorithm is therefore defined recursively.

Algorithm 5 HEAP-SEARCH($A, i, word$)

```

1: if  $i > A.heap-size$  then
2:   return  $-1$ 
3: end if
4: if  $A[i] = word$  then
5:   return  $i$ 
6: end if
7: if  $word < A[i]$  then
8:   return  $-1$ 
9: end if
10:  $j = \text{HEAP-SEARCH}(A, \text{LEFT}(i), word)$ 
11: if  $j \neq -1$  then
12:   return  $j$ 
13: end if
14: return  $\text{HEAP-SEARCH}(A, \text{RIGHT}(i), word)$ 

```

The initial call is

HEAP-SEARCH($A, 1, word$).

If the word is found, its array index is returned. Otherwise, the algorithm returns -1 .

For example, consider the heap

[apple, banana, orange, pear, grape].

Searching for **orange** returns index

3.

Searching for a word that does not occur in the heap, such as **mango**, returns

−1.

An important pruning case occurs when the target word is smaller than the current word. For example, suppose the algorithm reaches a node containing **orange** while searching for **mango**. Since

$\text{mango} < \text{orange}$,

and all descendants of **orange** must come after **orange**, the target cannot occur in that subtree. Therefore, the algorithm immediately returns −1 for that subtree.

Add a New Word

A new word should be inserted only if it is not already present in the priority queue. Therefore, the algorithm first calls HEAP-SEARCH.

If the word already exists, the algorithm does nothing. Otherwise, the new word is placed at the end of the heap.

Algorithm 6 MIN-HEAP-INSERT($A, word$)

```

1: if HEAP-SEARCH( $A, 1, word$ )  $\neq -1$  then
2:   return
3: end if
4:  $A.heap-size = A.heap-size + 1$ 
5:  $A[A.heap-size] = word$ 
6:  $i = A.heap-size$ 
7: while  $i > 1$  and  $A[PARENT(i)] > A[i]$  do
8:   exchange  $A[i]$  with  $A[PARENT(i)]$ 
9:    $i = PARENT(i)$ 
10: end while

```

The new word is initially inserted at the end of the heap. If the new word comes before its parent in dictionary order, the two words are exchanged. This process continues until the new word reaches a location where the min-heap property is satisfied or it becomes the root.

For example, suppose the heap is

[apple, banana, orange, pear, grape].

Suppose the new word is

aardvark.

The word is first placed at the end:

[apple, banana, orange, pear, grape, aardvark].

The parent of **aardvark** is **orange**. Since

$\text{aardvark} < \text{orange}$,

the two words are exchanged:

[apple, banana, aardvark, pear, grape, orange].

Now the parent of **aardvark** is **apple**. Since

aardvark < apple,

they are exchanged again:

[aardvark, banana, apple, pear, grape, orange].

The new word has reached the root, and the min-heap property is satisfied.

These algorithms preserve the min-heap property after every insertion and removal operation.

Task 3

Worst-Case Time Complexity Analysis

In this section, the worst-case running time of each operation in the heap-based priority queue is analyzed.

Let n denote the number of words currently stored in the heap. Since a binary heap is a complete binary tree, its height is $\Theta(\lg n)$.

Therefore, any operation that moves an element only along one path from the root to a leaf, or from a leaf to the root, takes at most $O(\lg n)$ time.

The operations considered are:

- HEAP-SIZE,
- MIN-HEAPIFY,
- MIN-HEAP-MINIMUM,
- MIN-HEAP-EXTRACT-MIN,
- HEAP-SEARCH, and
- MIN-HEAP-INSERT.

Complexity of Heap-Size

The number of words in the priority queue is stored directly in the variable **A.heap-size**.

The algorithm is HEAP-SIZE(A) and it simply returns $A.heap-size$.

No loop, recursion, or traversal is required. Therefore, the running time is constant.

$$T(n) = O(1)$$

Complexity of Min-Heapify

The MIN-HEAPIFY operation compares a node with its left and right children and, if necessary, exchanges the node with the smaller child.

At each level of the heap, only a constant number of operations are performed:

- find the left child,
- find the right child,
- compare the current word with the children,
- possibly exchange two words.

After an exchange, the algorithm continues recursively into only one of the two subtrees.

Since the height of a heap containing n elements is $O(\lg n)$, the algorithm can move downward through at most $O(\lg n)$ levels.

Therefore,

$$T(n) = O(\lg n)$$

in the worst case.

Complexity of Min-Heap-Minimum

In a min-heap, the minimum element is always stored at the root, which is located at index 1.

Therefore, the algorithm only returns $A[1]$.

No traversal is required.

Thus,

$$T(n) = O(1)$$

Complexity of Min-Heap-Extract-Min

The MIN-HEAP-EXTRACT-MIN operation performs the following steps:

1. obtain the minimum word from the root,
2. move the last word to the root,
3. decrease `A.heap-size`,
4. call MIN-HEAPIFY to restore the min-heap property.

The first three operations require constant time:

$$O(1).$$

The MIN-HEAPIFY operation requires

$$O(\lg n).$$

Therefore,

$$T(n) = O(1) + O(\lg n).$$

Since the logarithmic term dominates the constant term,

$$T(n) = O(\lg n)$$

in the worst case.

Complexity of Heap-Search

The heap is not completely sorted. The only ordering property is that each parent comes before its children in dictionary order.

Therefore, the fact that a word is not found in one subtree does not generally allow the other subtree to be ignored.

The search algorithm can sometimes stop early. For example, if

$$word < A[i],$$

then the target word cannot occur below node i , because all descendants of $A[i]$ must be greater than or equal to $A[i]$.

However, this pruning does not improve the worst-case running time.

Consider searching for a word that is larger than most of the words in the heap but is not present. In this situation, the condition

$$word < A[i]$$

may never become true, so the algorithm may have to visit every node in the heap.

If all n elements are examined, then

$$T(n) = cn + d$$

for some constants c and d .

Therefore,

$$T(n) = O(n)$$

in the worst case.

Complexity of Min-Heap-Insert

The insertion operation has two main parts.

First, the algorithm checks whether the word already exists by calling

$$\text{HEAP-SEARCH}(A, 1, word).$$

The worst-case running time of this search is

$$O(n).$$

If the word is not already present, it is added to the end of the heap. The new word may then move upward toward the root.

At each level, the algorithm compares the new word with its parent and may exchange the two words.

Since the height of the heap is

$$O(\lg n),$$

the upward movement requires at most

$$O(\lg n)$$

time.

Therefore, the total running time of insertion is

$$T(n) = T_{\text{search}}(n) + T_{\text{heap insertion}}(n).$$

Substituting the two costs gives

$$T(n) = O(n) + O(\lg n).$$

Since

$$O(n)$$

grows faster than

$$O(\lg n),$$

the linear term dominates.

Therefore,

$$\boxed{T(n) = O(n)}$$

in the worst case.

It is important to distinguish this operation from ordinary heap insertion. If duplicate checking were not required, heap insertion alone would require only

$$O(\lg n).$$

However, here requires a word to be inserted only if it is not already present. Therefore, the search for duplicates makes the complete insertion operation $O(n)$.

Summary of Worst-Case Complexities

The worst-case running times of the operations are summarized below.

Operation	Worst-Case Time	Reason
HEAP-SIZE	$O(1)$	Directly returns <code>A.heap-size</code>
MIN-HEAP-MINIMUM	$O(1)$	Minimum word is stored at the root
MIN-HEAPIFY	$O(\lg n)$	Moves down at most the height of the heap
MIN-HEAP-EXTRACT-MIN	$O(\lg n)$	Calls MIN-HEAPIFY
HEAP-SEARCH	$O(n)$	May examine all n words
MIN-HEAP-INSERT	$O(n)$	$O(n)$ search plus $O(\lg n)$ heap insertion

Table 1: Worst-case time complexity of the heap operations.

Final Analysis

The heap structure is efficient for operations involving the first word in dictionary order. Accessing the first word takes constant time, and removing it takes logarithmic time because only one path through the heap may need to be repaired.

Searching for an arbitrary word is less efficient because a heap is only partially ordered. In the worst case, all words must be examined, giving a running time of $O(n)$.

The insertion operation would normally take $O(\lg n)$ for a heap. However, because this problem requires checking whether the new word is already present before inserting it, the complete insertion operation has worst-case complexity

$$O(n).$$

Task 4

Implementation

The complete implementation of the heap-based priority queue is provided in the separate source-code file

`heap_based_priority_queue.cpp`.

The program implements the binary min-heap data structure and all operations described in Task 2. The main components are:

- reporting the current number of words,
- searching for a word and returning its index or -1 ,
- inserting a new word if it is not already present,
- removing the first word in dictionary order,
- restoring the min-heap property using MIN-HEAPIFY, and
- displaying the heap for testing purposes.

The implementation uses a C++ `vector<string>` to store the heap and an integer variable `heapSize` to keep track of the number of words currently stored.

Index 0 of the vector is left unused so that the implementation uses the same 1-based heap indexing as the algorithms described in Task 2.

The implementation does not use the C++ built-in `priority_queue`.

Task 5

Correctness Testing

The implementation was tested using a sequence of operations that covers all required functions of the heap-based priority queue. The complete sample input and output are also provided separately in the file

`sampleInputOutput.txt`.

The test includes insertion, size checking, searching, duplicate detection, removal of words, heap restoration, and removal from an empty priority queue.

Sample Input

The following input was used:

```
20
ADD pear
ADD apple
ADD orange
ADD banana
ADD grape
SIZE
PRINT
SEARCH orange
SEARCH mango
ADD apple
SIZE
REMOVE
PRINT
REMOVE
PRINT
REMOVE
REMOVE
REMOVE
REMOVE
SIZE
```

The first value, 20, indicates that the program should process 20 operations.

Sample Output

The corresponding output is:

```
Added: pear
Added: apple
Added: orange
Added: banana
Added: grape
Size: 5
Heap: [1]=apple [2]=banana [3]=orange [4]=pear [5]=grape
Search orange: 3
Search mango: -1
apple already exists
Size: 5
Removed: apple
Heap: [1]=banana [2]=grape [3]=orange [4]=pear
Removed: banana
Heap: [1]=grape [2]=pear [3]=orange
Removed: grape
Removed: orange
Removed: pear
Queue is empty
Size: 0
```

Analysis of the Test Results

The first five operations insert the words `pear`, `apple`, `orange`, `banana`, and `grape` into the priority queue.

After the five insertions, the command

```
SIZE
```

```
produces
```

```
Size: 5
```

which confirms that the `heapSize` variable is updated correctly during insertion.

The first `PRINT` operation produces

```
Heap: [1]=apple [2]=banana [3]=orange [4]=pear [5]=grape
```

The word `apple` is stored at the root because it is the first word in dictionary order. The parent-child relationships also satisfy the min-heap property.

For example,

$$\text{apple} < \text{banana},$$
$$\text{apple} < \text{orange},$$
$$\text{banana} < \text{pear},$$

and

$$\text{banana} < \text{grape}.$$

Therefore, the heap after insertion is a valid min-heap.

Testing Search

The command

```
SEARCH orange
```

```
produces
```

```
Search orange: 3
```

which shows that the search operation successfully finds an existing word and returns its heap index.

The command

```
SEARCH mango
```

```
produces
```

```
Search mango: -1
```

which confirms that the search operation correctly returns -1 when the requested word is not present in the priority queue.

Testing Duplicate Insertion

The word **apple** is then added again using

```
ADD apple
```

Since **apple** is already present, the output is

```
apple already exists
```

and the word is not inserted again.

The following **SIZE** operation still produces

```
Size: 5
```

which confirms that duplicate words are not added to the data structure.

Testing Removal

The first **REMOVE** operation produces

```
Removed: apple
```

This is correct because **apple** is the first word in dictionary order and was stored at the root of the min-heap.

After removing **apple**, the program prints

```
Heap: [1]=banana [2]=grape [3]=orange [4]=pear
```

The new root is **banana**, which is now the first word in dictionary order. The heap still satisfies the min-heap property.

The next removal produces

```
Removed: banana
```

and the heap becomes

```
Heap: [1]=grape [2]=pear [3]=orange
```

Again, **grape** becomes the root because it is the first remaining word in dictionary order.

The remaining removal operations produce

```
Removed: grape
```

```
Removed: orange
```

```
Removed: pear
```

Therefore, the complete order in which the five original words are removed is

apple, banana, grape, orange, pear.

This sequence is exactly the dictionary order of the five inserted words. This confirms that the min-heap correctly maintains the required priority ordering.

Testing an Empty Priority Queue

After all five words have been removed, another `REMOVE` operation is performed.

The program produces

`Queue is empty`

instead of attempting to access an invalid heap position. This confirms that the implementation correctly handles removal from an empty priority queue.

Finally, the last `SIZE` operation produces

`Size: 0`

which confirms that the priority queue is empty and that `heapSize` was correctly updated after all removal operations.

Based on these test results, all required operations of the heap-based priority queue work correctly for the given test case.

AI use Acknowledgment

Generative AI (ChatGPT) was used to assist with LaTeX formatting, some part of writing and presentation of this report.